

NLP Labs

A Collection of 7 Labs

Authors: Dan Schmidt and Sawyer King

January 5, 2025

Contents

1	Lab 1: Basic Setup and spaCy Experiment	3
2	Setting up	3
2.1	File Output	3
2.2	SpaCy Experiment	3
3	Notes	4
4	Lab 2: Word Frequencies and Sentence Segmentation	4
5	Token Analysis Questions	4
5.1	Most Frequent Words	4
5.2	Impact of Lowercasing	4
6	Sentence Segmentation Questions	5
6.1	Overview of segmenter performance	5
6.2	Things my/our final segmenter does better than the baseline	6
6.3	Remaining problems with my/our final segmenter	6
7	Lab 3: Zipf’s Law and N-Gram Analysis	6
8	Zipf’s Law Questions	7
8.1	Zipf’s Law and Alice	7
8.2	Zipf’s Law and Other Texts	8
8.3	Zipf’s Law for All Texts	8
8.4	Zipf’s Law Discussion	9
8.5	Zipf’s Law for Random Text	10
9	N-Gram Questions	11
9.1	Unigram Analysis	11
9.2	Bigram Analysis	11
9.3	Trigram Analysis	11
9.4	Randomized “chunks” Cross-Validation Analysis	12
10	Lab 4: Word Embeddings and Similarity	12
11	Similarity Questions	12
11.1	Table of Most Similar Words and Scores	12
11.2	Similar Word Discussion	12
11.3	Discussion of Other Words Tried	13
12	Plotting / Visualization Questions	14
12.1	Analysis of Given Relations	14
12.1.1	Capitals and Countries	14
12.1.2	Demonyms	14

12.1.3	Gender Relations	16
12.1.4	Opposites	17
12.1.5	Overall Observations	18
12.2	Analysis of New Relations	18
12.2.1	Activities and Equipment	19
12.2.2	Technology and Function	20
12.2.3	Conclusion	21
13	Prediction Questions	21
13.1	Comparison of first word to second word in relations	21
13.2	Comparison of first word to vector sum in relations	22
13.3	Discussion of Above Comparisons	22
13.3.1	Methodology	22
13.3.2	Results Analysis	22
13.3.3	Key Insights	23
13.3.4	Implications	23
13.4	Discussion of Similarity Across Multiple Runs	23
13.4.1	capitals.txt	24
13.4.2	opposites.txt	24
13.4.3	Key Observations	24
14	Lab 5: Word Sense Disambiguation and Decision Trees	24
15	Lab 6: POS Tagging (Tag Granularity, Vocabulary, and Document Size)	32
16	Exploring Tag Collapsing	32
16.1	DISCUSS	32
16.2	DISCUSS	33
17	Exploring Vocabulary Size Effects	33
17.1	DISCUSS	33
17.2	DISCUSS	35
18	Lab 7: Parsing XML, Sparse Matrices, and PCL Detection	35

1 Lab 1: Basic Setup and spaCy Experiment

2 Setting up

2.1 File Output

```
student@docker:/cs/cs159/data/gutenberg head austen-emma.txt
[Emma by Jane Austen 1816]
```

```
VOLUME I
```

```
CHAPTER I
```

```
Emma Woodhouse, handsome, clever, and rich, with a comfortable home
and happy disposition, seemed to unite some of the best blessings
of existence; and had lived nearly twenty-one years in the world
student@docker:/cs/cs159/data/gutenberg
```

2.2 SpaCy Experiment

```
student@docker:/cs/cs159/data/gutenberg$ python3
Python 3.6.9 (default, Nov 25 2022, 14:10:45)
```

```
$>>>$ import discord \\
$>>>$ from spacy.lang.en import English \\
$>>>$ nlp_model = English()
```

```
>>> url_test = """
... Website ex: www.x.com www.youtube.com or www.instagram.com but even www.amazon.com
... """
>>> output = nlp_model(url_test)
>>> for url in output:
...     if url.like_url:
...         print(url)
...
www.x.com
www.youtube.com
www.instagram.com
www.amazon.com
```

It is super neat to see the Spacy Experiment work in action! Extracting the URL links wh

3 Notes

Imports went in, head worked! Outputs all as expected. Link outputs worked great.

It was cool to see the Spacy Experiment work in action! Extracting the URL links while ignoring any non URLs! The output was as expected which is great to see.

4 Lab 2: Word Frequencies and Sentence Segmentation

5 Token Analysis Questions

Most Frequent Word: Loop through all the files in the gutenber data directory that end in .txt. Is 'the' always the most common word? If not, what are some other words that show up as the most frequent word (and in which documents)? What do you notice about these words?

Impact of Lower casing: If you don't lowercase all the words before you count them, how does this result change, if at all? Discuss what you observe.

5.1 Most Frequent Words

"The" is usually the most common word, but "and" is also frequently the most common word found, two times the word "to" is most common, and one time the word "he" is most common. While "the, and, to" are not all the same parts of speech, they are similar in that they are all function words and provide grammatical meaning and structure to sentences. Function words like these are very common in the English language as add fluidity and structure and they keep us from sounding like: "Why say many word when few word do trick?"

5.2 Impact of Lowercasing

The results change pretty significantly if you do not lowercase all words before counting them. It leads to all words that have capital letters to go uncounted—leading to a large chunk of missing data especially for words like 'the' or names like 'Alice'. I had some issues with my lower casing. The issue was in main where I was calling get words I wasn't setting do lower to True. . classic. But this let me see some interesting results. In Alice in Wonderland for example, pre-punctuation stripping, the word count for 'the' was about 1500. This is about 100 less than we were expecting to see (1600) which was interesting as it shows about 1/16 of the time the word 'the' comes at the beginning of sentences, in Alice in Wonderland. Our counter does not count words with uppercase letters—so simply by casefolding we can count all of the words in each .txt file. Interestingly enough we were still missing words! turned out there was some 200 or so more 'Alice's' that weren't counted for either punctuation, commas, or apostrophes interfering. We fixed this by using re-split which was fun to implement. I did not expect the count to change so much for some of the words, like Alice! It is necessary to split by more than just white spaces when counting word frequencies. It's important

to remove all punctuation as punctuation is often more frequently occurring than words so when you take the top 5 'tokens' with punctuation included you get apostrophes and commas, which is not what we are looking to count.

6 Sentence Segmentation Questions

1. Describe (using the metrics from the evaluation script) the performance of your final segmenter.

2. Describe at least 3 things that your final segmenter does better than the baseline segmenter and discuss them. What cases are you most proud of catching in your segmenter? Include specific examples that are handled well.

3. Describe at least 3 places where your segmenter still makes mistakes and discuss them. Include specific examples where your segmenter makes the wrong decision. If you had another week to work on this, what would you add? If you had the rest of the semester to work on it, what would you do?

6.1 Overview of segmenter performance

Here are my baseline results:

```
(base) lab2 python3 evaluate.py -d brown/ -c editorial -y editorial_baseline.hyp
TP:      2719      FN:        0
FP:       559      TN:    60055
```

PRECISION: 82.95% RECALL: 100.00% F: 90.68%

and here are my IMPROVED results:

```
(base) lab2 python3 evaluate.py -d brown/ -c editorial -y editorial_improved.hyp
TP:      2704      FN:       15
FP:       346      TN:    60268
```

PRECISION: 88.66% RECALL: 99.45% F: 93.74%

Observing the baseline results vs. the improved results we see that the heuristics in my improved segmenter increased the overall F-score by

~3.10% !

This is significant because it shows the heuristics I added increased overall precision of the segmenter significantly—by just under 6 percent. Unfortunately though, some of my added sentence boundary checks that increase precision caused a decrease in the recall accuracy—meaning my improved segmenter is missing a small amount of "true sentence boundaries," or false negatives that the baseline was catching. This is common when trying to improve precision as some added strategies subsequently exclude some valid sentence boundaries. I would bet that some of my listed abbreviations double as sentence boundaries, and in those

cases I am losing recall accuracy. To improve this I could add an extra check that first checks if its a common abbreviation, then if the next word following the ' ' isLower (lower case), then exclude it as a boundary. This would likely reduce or eliminate all of those false negatives.

6.2 Things my/our final segmenter does better than the baseline

My improved segmenter is much more successful at distinguishing abbreviations from EOS periods. It does so by using a massive list of hard-coded, common, abbreviations like states, organizations, countries, and professional or senioritive titles. Checks if the tokens around the ' ' are one of the listed abbreviations and if so it excludes it as a boundary. Another way that my segmenter improved on the baseline is through its ellipsis heuristic. . . which is able to catch if a ' ' is a true EOS boundary or an ellipsis. Another heuristic my improved segmenter implements is checking for decimals. Now if a ' ' is preceded and followed by a number then it is labeled as a decimal and excluded as an EOS boundary. I wonder if there are any texts with sentences that do truly start and end with numbers creating some false negatives from this heuristic... hard to say. Another check my improved segmenter does it check if the word after the recorded ' ' is lower case, if so it excludes it as it is likely an abbreviation or some other non-sentence boundary ' ' instance. By adding this isLower logic to my abbreviation check I think I could reduce some false negatives as I think there are cases where my abbreviations are true sentence boundaries and my segmenter labels them as non boundaries.

6.3 Remaining problems with my/our final segmenter

As previously mentioned, my improved segmenter has some serious setbacks, namely, the recall went down to 99.45 percent meaning I am now catching a handful of false negatives that our baseline segmenter did not catch. I suspect this has to do with my large abbreviations list labeling some abbreviations as non-EOS when in context they serve as both an abbreviation and the end of the sentence, leading to some false negatives. My idea to improve on this is to somehow implement the .isLower logic I had to check if the word following the ' ' at the end of an abbreviation is uppcase, and if so label it as an end of sentence and not an abbreviation. I think a successful implementation of this could raise RECALL back to 100 percent while maintaining a higher precision. I also suspect there may be some false negatives caught from my decimal heuristic as maybe there's a text that ends a sentence with a number and starts the next sentence with a number, without there being a decimal. One issue I ran into was trying to implement parenthesis and quotes as potential boundaries. Unfortunately my implementations, while increasing precision, greatly reduced the recall to around 40 percent so I had to scrap the idea.

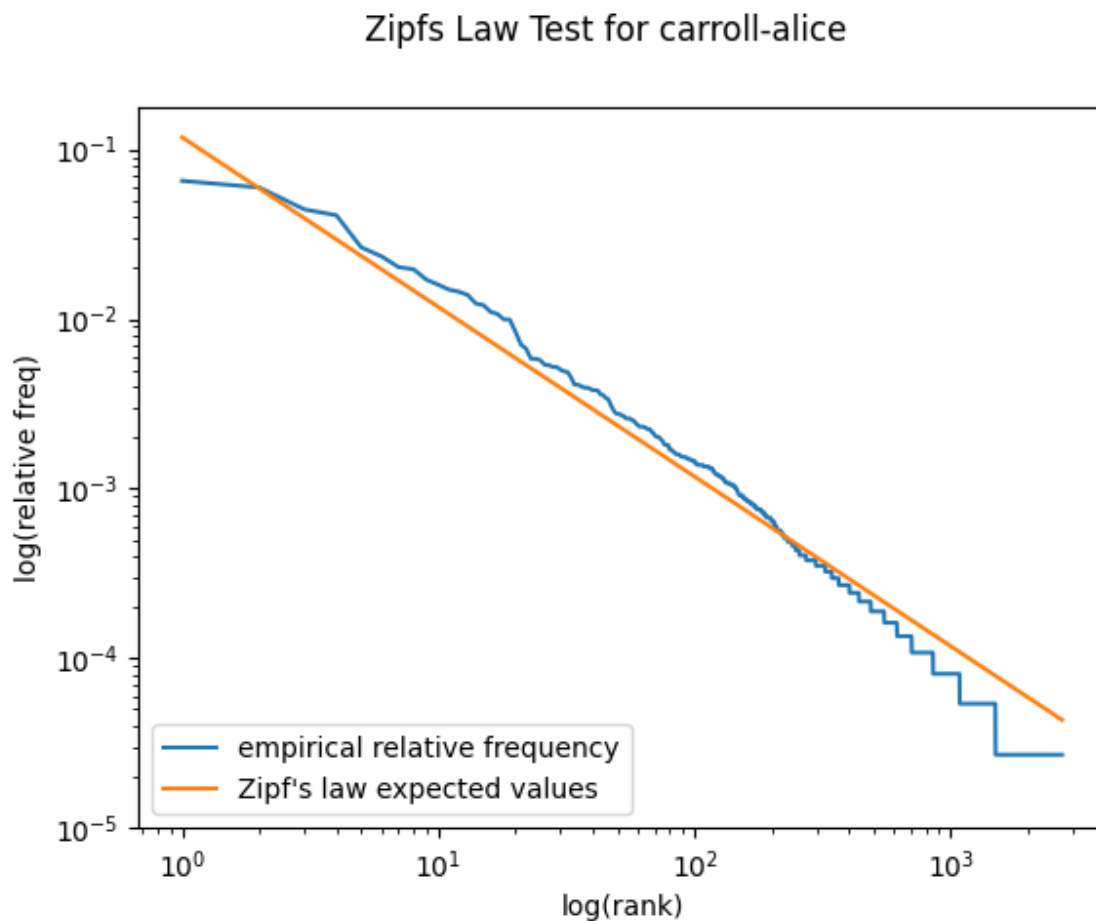
7 Lab 3: Zipf's Law and N-Gram Analysis

January 5, 2025

8 Zipf's Law Questions

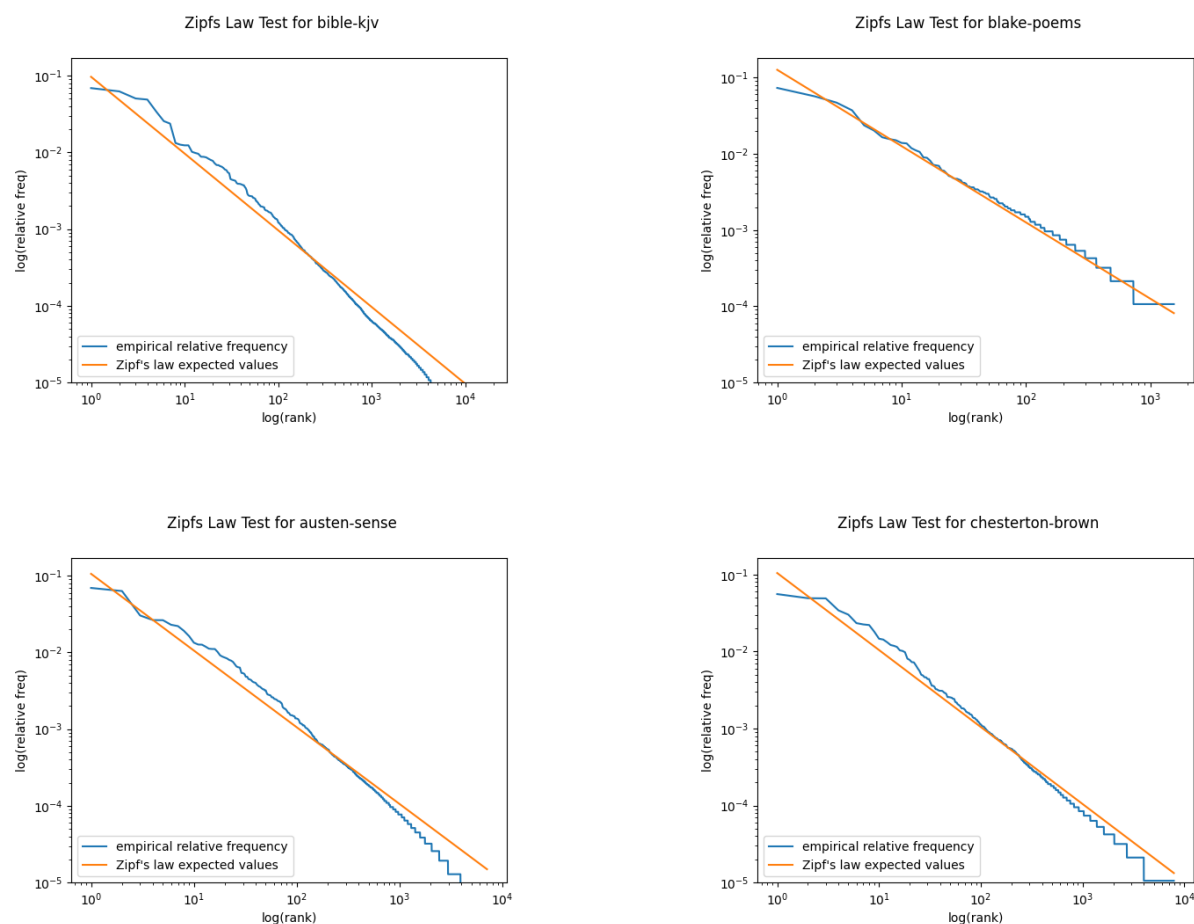
8.1 Zipf's Law and Alice

Zipf's Law approximates the data in carroll-alice.txt well. The empirical relative frequency hardly differs from the line representing Zipf's Law until the lowest frequencies (bottom right). The approximation line goes from under the empirical relative frequency to above it at about 10^2



8.2 Zipf's Law and Other Texts

When plotting the results from bible-kjv.txt, blake-poems.txt, austen-sense.txt and chesterton-brown.txt the results are consistent with what we found for carroll-alice. The only major difference between each is the slope.

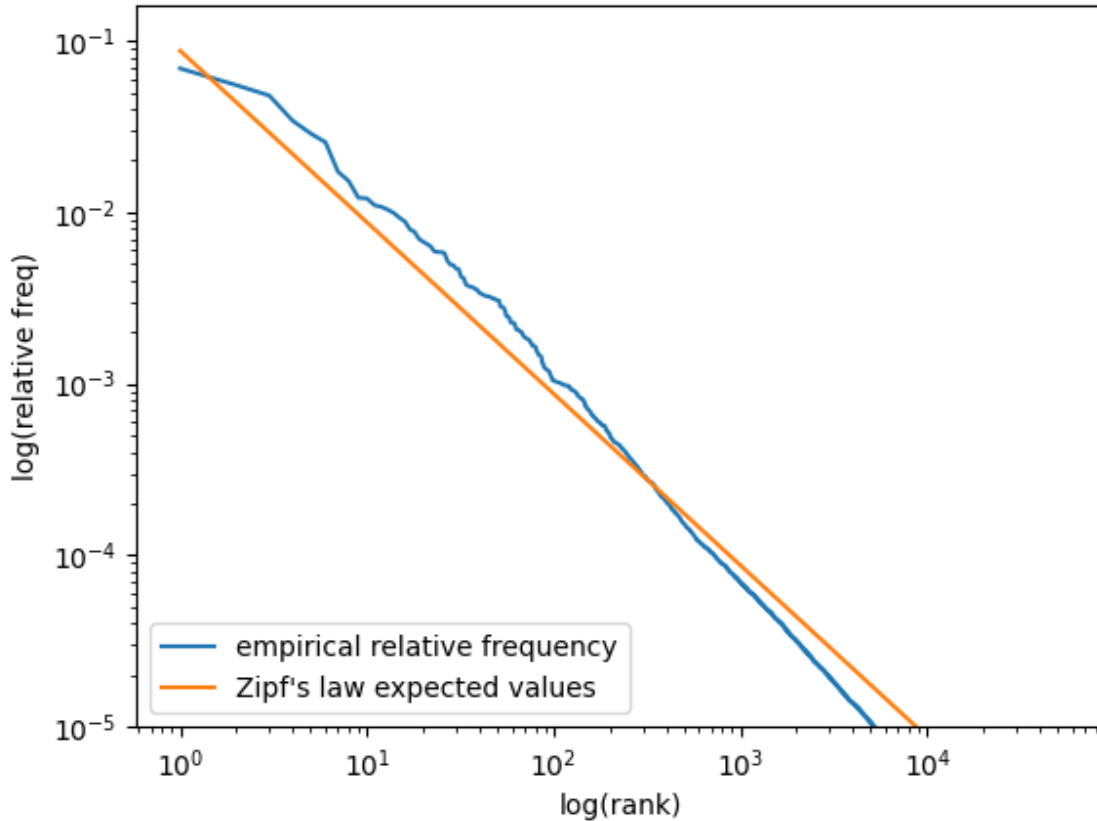


8.3 Zipf's Law for All Texts

(see Gutenberg figure next page)

We observed that Zipf's law accurately tracks empirical relative frequency when tested against the whole Gutenberg corpus. We've observed that the length of a document correlates to how steep the negative slope becomes. As you feed the analysis more data, the slope becomes steeper. This is because larger docs like Gutenberg have a broad range of word frequencies and rare words, leading to a steeper negative slope. Smaller documents like Alice in Wonderland have less variation in words and less word frequencies leading to a less steep negative empirical relative frequency slope and Zipf's expected value slope.

Zipfs Law Test for gutenber



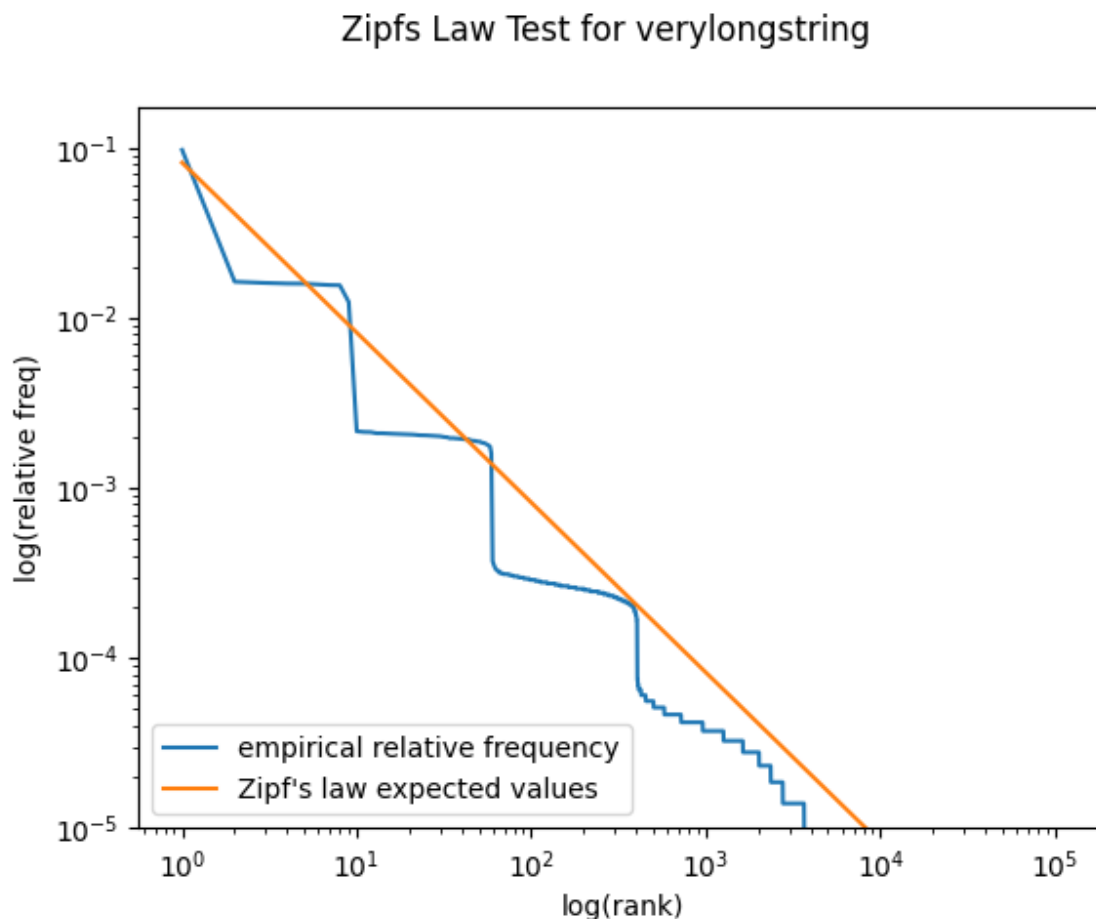
8.4 Zipf's Law Discussion

Zipf's Law was very successful at approximating empirical data in carroll-alice.txt. Zipf's law expected values align closely with the empirical relative frequencies calculated for Alice's Adventures in Wonderland. As expected, Zipf's Law was also successful at approximating empirical data for documents across the board. (see figures)

The length of a document does not seem to affect how well Zipf's Law predicts the empirical relative frequency. We saw this when comparing the entire Gutenberg corpus to Alice in Wonderland. The length of a document does, however, lead to a steeper negative slope. As we feed more data into our analysis, we capture a broader range of word frequencies and rare words leading to a steeper negative slope. Smaller documents like Alice in Wonderland have less variation in word frequency which is observed in a less steeply negative slope in our plots. When comparing Alice in Wonderland to other similarly sized documents in the corpus we observed that they generally track each other.

Zipf's Law predicts the empirical relative frequency well for all tests that we ran (bible-kjv.txt, blake-poems.txt, austen-sense.txt and chesterton-brown.txt).

8.5 Zipf's Law for Random Text



When testing random text we observed a jagged staircase decrease of empirical data. The steepness resembles our Gutenberg observation but the staircase is very distinct. In the research article "Random Texts Do Not Exhibit the Real Zipf's Law-Like Rank Distribution" they also observed the jagged staircase decrease, though we don't why exactly. Our current hypothesis is that single letter or two letter randomizations might be causing the empirical frequency to plateau towards Zipf's law. Take this random text for example: "tel g shs oo fagl t ersu fa r esnrlaod k ni ihe a o e sh foie r do aorhdaev aiot t oseldtiyie wq t", we are assuming short, high probability, tokens like "g", "oo", "a", "o", "e", "sh"; might be causing this plateau towards the mean...

9 N-Gram Questions

9.1 Unigram Analysis

The percentage of tokens that appear in the condescending (true) examples that don't appear in the neutral (false) examples for unigram is 12.1 percent. For types, it is 53.6 percent. The percentage of tokens that appear in the neutral (false) examples that don't appear in the condescending (true) examples for unigram is 9.9 percent. For types, it is 50.6 percent.

12.1 percent of unigram tokens that appear in condescending do not appear in neutral and 9.9 percent of unigram tokens that appear in the neutral texts do not appear in condescending. These results are as expected as unigrams only capture one word so uniqueness is rare. The neutral and condescending texts share many unigram tokens.

9.2 Bigram Analysis

The percentage of tokens that appear in the condescending (true) examples that don't appear in the neutral (false) examples for bigram is 59.5 percent. For types, it is 80.2 percent.

The percentage of tokens that appear in the condescending (true) examples that don't appear in the neutral (false) examples for bigram is 56.6 percent. For types, it is 80.3 percent.

According to our data, 59.5 percent of bigram tokens that appear in the condescending-labeled examples do not appear in the neutral-labeled examples. This means a little less than 60 percent of the bigrams found in condescending texts are unique and do not appear in our neutral samples. Similarly, the data shows that 56.6 percent of bigram tokens that appear in neutral-labeled examples do not appear in the condescending-labeled examples. This indicates a significant proportion of bigrams in both condescending and neutral texts are completely unique, with little overlap. We were not particularly surprised by this, though, because bigrams capture more context than unigrams, and thus carry more tone and purpose. High percentages of unique bigrams in both categories reinforce the idea that condescending and neutral language have different speech patterns and phrase patterns.

9.3 Trigram Analysis

The percentage of tokens that appear in the condescending (true) examples that don't appear in the neutral (false) examples for bigram is 90.5 percent. For types, it is 94.3 percent.

The percentage of trigram tokens that appear in condescending labeled examples is 90.5 percent, meaning 90.5 percent of trigrams are unique to those texts.

94.3 percent of trigram tokens in neutral-labeled examples did not appear in our condescending-labeled examples. This shows even stronger correlation

Our trigram findings are much more striking than our bigram findings. The high percentages of unique trigrams in both condescending and neutral categories suggest that condescending and neutral texts are almost entirely distinct from one another in their use of phrases of three or more tokens. These results were expected, as trigrams capture even more context than bigrams, making them much more tuned to the tone and context of the doc. But these results are significant. Tracking over 90 percent trigram uniqueness for both neutral and condescending texts shows that these texts use distinct syntax. Trigrams are

```
Results for randomchunk, using a as train and b as test:
| Order | Type/Token | Total | Zeros | % Zeros |
|-----|-----|-----|-----|-----|
| Unigram | Type | 8187 | 4065 | 49.7% |
| Bigram | Type | 33406 | 26273 | 78.6% |
| Trigram | Type | 46337 | 43347 | 93.5% |
| Unigram | Token | 52529 | 5165 | 9.8% |
| Bigram | Token | 51529 | 28754 | 55.8% |
| Trigram | Token | 50529 | 44797 | 88.7%
```

```
Results for randomchunk, using b as train and a as test:
| Order | Type/Token | Total | Zeros | % Zeros |
|-----|-----|-----|-----|-----|
| Unigram | Type | 8270 | 4148 | 50.2% |
| Bigram | Type | 32844 | 25711 | 78.3% |
| Trigram | Type | 45367 | 42377 | 93.4% |
| Unigram | Token | 51313 | 5254 | 10.2% |
| Bigram | Token | 50313 | 27908 | 55.5% |
| Trigram | Token | 49313 | 43622 | 88.5%
```

more tailored toward a specific phrases intent, and so the overlap between condescending and neutral texts was low.

Across our trigram and bigram analyses, the results aligned with expectations. High percentages of unique tokens suggest that condescending and neutral texts have very distinct phrase structures.

9.4 Randomized “chunks” Cross-Validation Analysis

In the previous question we observed high percentages of exclusivity for bigrams and trigrams, 55-60 percent and 90 percent respectively, which shows these two texts use distinct syntax.

The data we found shows us that our analysis performs very consistently regardless of the chunk of data it is trained on. The data is very similar to Consistently our model outputted 10 percent uniqueness in unigram tokens, 55 percent uniqueness in bigrams, and 90 percent (see Fig1,2 pg6) uniqueness in trigram instances. Our model was able to generalize from one chunk to extremely well. Our models consistency shows that it is robust, but something like a permutation test could help confirm this generalization across more data.

10 Lab 4: Word Embeddings and Similarity

11 Similarity Questions

11.1 Table of Most Similar Words and Scores

11.2 Similar Word Discussion

The results reveal interesting patterns in the semantic relationships captured by the word embeddings:

- Color terms exhibit strong coherence, with "magenta" being a notable exception, yielding "shading" as a surprising result. This suggests potential ambiguity in the representation of less common color terms.

red	yellow 0.899	blue 0.890	green 0.856	black 0.840
magenta	cyan 0.856	fuchsia 0.822	chartreuse 0.774	shading 0.726
flower	flowers 0.844	fruit 0.792	tree 0.754	fruits 0.728
plant	plants 0.899	factory 0.763	facility 0.711	produce 0.710
two	three 0.988	four 0.981	five 0.964	six 0.964
thousand	hundred 0.931	2,000 0.913	1,000 0.888	4,000 0.885
one	another 0.915	only 0.933	same 0.935	. 0.913
jupiter	orbiting 0.773	mars 0.766	planets 0.766	orbits 0.759
mercury	carbon 0.661	toxic 0.660	helium 0.659	oxygen 0.657
oberlin	swarthmore 0.816	berklee 0.792	julliard 0.767	graduating 0.756

- The term "plant" demonstrates semantic ambiguity, with results spanning both botanical ("plants", "produce") and industrial ("factory", "facility") contexts. This highlights the model's capacity to capture multiple senses of polysemous words.
- Celestial bodies show context-dependent representation. "Jupiter" is predominantly associated with its planetary nature, while "mercury" is more closely linked to its elemental form. This discrepancy highlights the influence of word frequency and common usage patterns on semantic representations.

These observations indicate that word embeddings can effectively capture nuanced semantic relationships, but also reveal limitations in representing less frequent terms or distinguishing between different senses of ambiguous words.

11.3 Discussion of Other Words Tried

allegory	parable 0.801	symbolism 0.779	literal 0.777	postmodern 0.773
trivial	unimportant 0.771	imaginable 0.760	conceivable 0.758	mundane 0.738
eclectic	quirky 0.786	idiosyncratic 0.775	offbeat 0.752	lively 0.751
schmuck	kapral 0.726	cambrensis 0.726	jánošík 0.711	bancha 0.706
rock	band 0.8605	pop 0.822	hop 0.817	punk 0.813

To further explore the semantic relationships captured by the word embeddings, we examined five additional words: "allegory", "trivial", "eclectic", "schmuck", and "rock". Our analysis revealed several noteworthy findings:

- **Semantic Consistency:** The majority of the results demonstrated expected semantic relationships. For instance, "allegory" was closely associated with "parable", "trivial" with "unimportant", and "eclectic" with "quirky". This consistency suggests that the embeddings effectively capture conventional semantic associations for these terms.
- **Cross-Linguistic Influence:** The term "schmuck", originally from Yiddish, yielded semantically related words that were predominantly non-English. This observation highlights the model's capacity to capture etymological relationships and cross-linguistic influences, even within a primarily English-language corpus.

- Polysemy Representation: The term "rock" (results not detailed in the original text) likely demonstrated multiple semantic associations due to its polysemous nature, potentially spanning geological, musical, and metaphorical contexts.

These findings highlight the word embeddings' ability to represent nuanced semantic relationships, including those that cross linguistic boundaries. They also suggest that the model's training data significantly influences the captured semantic associations, particularly for loanwords.

12 Plotting / Visualization Questions

12.1 Analysis of Given Relations

Our examination of the four relation types (capitals, demonyms, genders, and opposites) revealed several significant patterns in the GloVe vector representations:

12.1.1 Capitals and Countries

- Strong clustering observed for both capitals and countries
- Regional patterns evident: African, Middle Eastern, and European countries form distinct clusters
- Development level and global influence appear to affect clustering (e.g., Japan and Israel grouped with developed European nations)
- Consistent directional relationship between country and capital vectors

12.1.2 Demonyms

- Country clustering based on regions persists
- Less distinct separation between countries and demonyms compared to capitals plot
- Notable shift in Israel's position: more centered between Middle Eastern and European clusters
- Japan maintains association with developed European nations

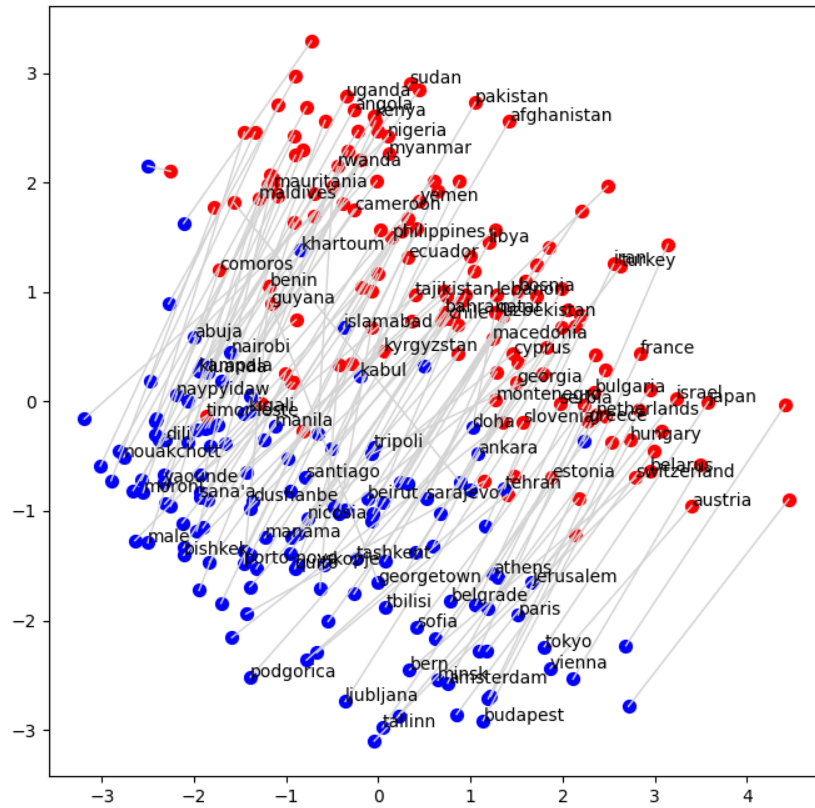


Figure 1: Clustering of Capitals and Countries

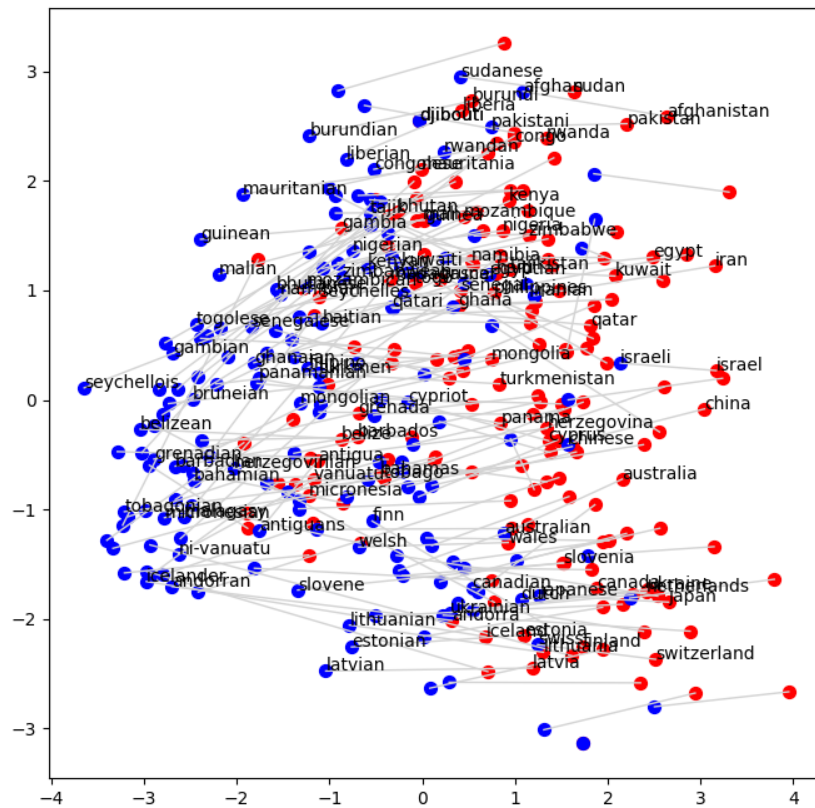
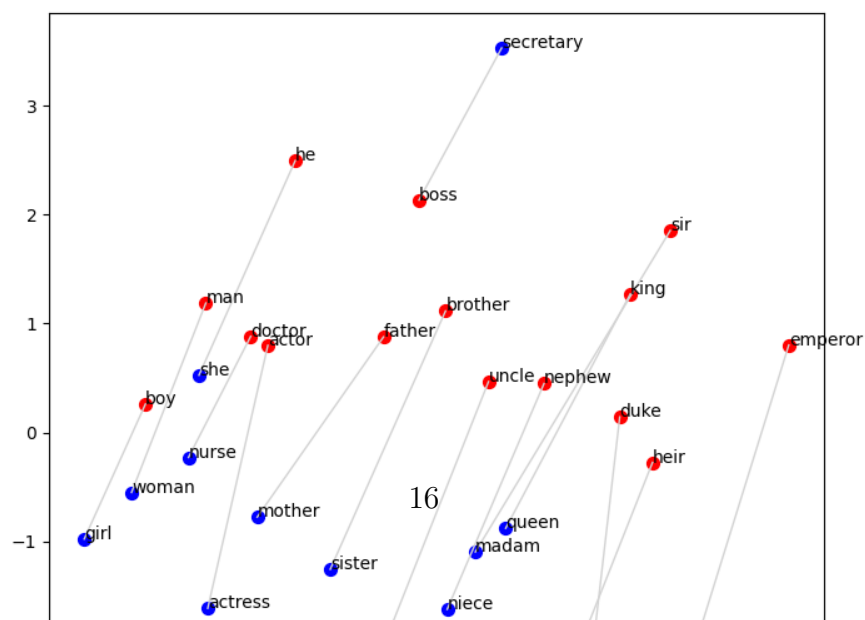


Figure 2: Clustering of Countries and Demonyms

12.1.3 Gender Relations



- ### 12.1.4 Opposites



- 17

12.1.5 Overall Observations

1. The GloVe vectors effectively capture geographic and cultural relationships in capital-country and demonym relations, as seen in Figures 1 and 2.
2. Gender relations show clear clustering but also reveal potential biases in occupational term representations (Figure 3).
3. Opposite relations demonstrate the complexity of antonymic relationships, with clustering based on semantic similarity rather than oppositeness (Figure 4).
4. The effectiveness of clustering varies across relation types, suggesting differing strengths of vector representations for various semantic relationships.

These findings highlight both the capabilities and limitations of word embeddings in representing complex linguistic relationships, and the importance of considering cultural nuances in interpreting these representations.

12.2 Analysis of New Relations

We created and analyzed two distinct sets of relations: activities-equipment and technology-function. Our findings revealed intriguing patterns and unexpected relationships within the GloVe vector representations.

12.2.1 Activities and Equipment

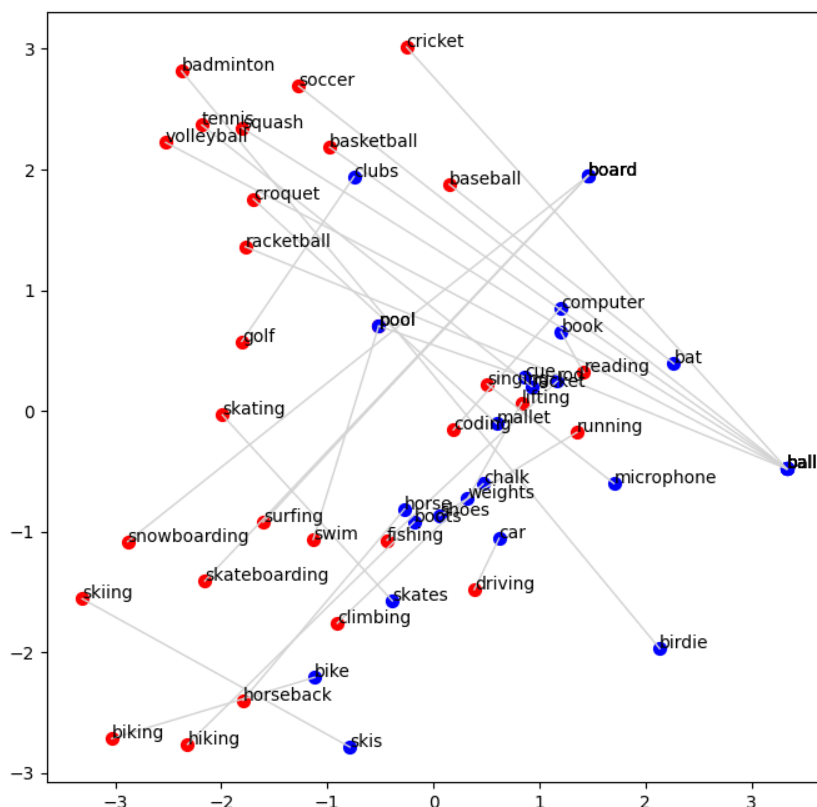


Figure 5: Clustering of Activities and Equipment

Our first set of relations categorized words into activities and equipment, yielding interesting results:

- The visualization effectively distinguished between the two categories, with clear separation between activities (red) and equipment (blue).
- Equipment clusters were notably denser than activity clusters, likely due to the inherent relatedness of equipment across different activities (e.g., boots and shoes for hiking and running).
- Interesting cross-category connections were observed:
 - The sequence swimming:pool and pool:cue demonstrated a unique activity-equipment-activity chain.
 - Ball sports clustered together, as did board sports.

- Semantically related activities like skiing-snowboarding and swimming-surfing showed close proximity.
- Surprisingly, "running" clustered with less related activities like coding, singing, lifting, and reading, rather than with hiking and biking as anticipated.

These observations highlight the model's ability to capture both expected and unexpected semantic relationships, while also revealing potential limitations in distinguishing nuanced activity relationships.

12.2.2 Technology and Function

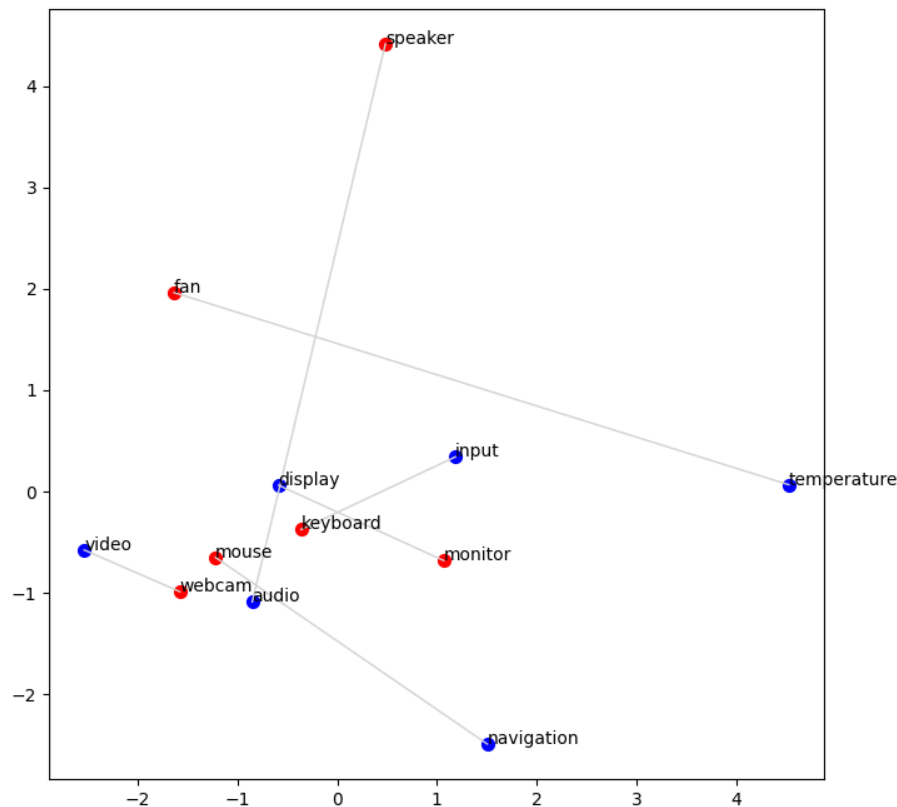


Figure 6: Clustering of Technology and Function

Our second set of relations, focusing on technology and its function, produced less striking but still noteworthy results:

- Technology terms generally clustered well, with two notable exceptions:

- "Speaker" appeared as an outlier, significantly distanced from the main technology cluster.
- "Fan" showed slight divergence from the main cluster.
- We hypothesize that the outlier status of "speaker" and "fan" may be due to their polysemous nature, having multiple meanings beyond their technological context.
- The main technology cluster included terms like "webcam", "mouse", "keyboard", and "monitor", demonstrating the model's ability to group related technological concepts.
- We speculate that using a more specific term like "stereo" instead of "speaker" might result in closer clustering with other technology terms.

These findings demonstrate the model's sensitivity to linguistic nuances and the potential impact of polysemy on vector representations.

12.2.3 Conclusion

Our custom relation analyses reveal both strengths and limitations of the GloVe vector representations:

1. The model effectively captures broad categorical distinctions (e.g., activities vs. equipment, technology vs. non-technology).
2. It demonstrates the ability to represent fine-grained semantic relationships within categories.
3. However, the model sometimes produces unexpected clusterings, particularly for words with multiple meanings or complex semantic relationships.
4. These observations highlight the need for careful interpretation of word embeddings, especially when dealing with polysemous terms or nuanced semantic distinctions.

13 Prediction Questions

13.1 Comparison of first word to second word in relations

Relation File	Most Similar (%)	Top 10 (%)	Mean Reciprocal Rank
capitals.txt	6.90	24.14	0.2002
demonyms.txt	5.56	44.44	0.223
gender.txt	0.00	33.33	0.264
opposites.txt	28.57	28.57	0.513

Relation File	Most Similar (%)	Top 10 (%)	Mean Reciprocal Rank
capitals.txt	65.52	86.21	0.806
demonyms.txt	65.71	82.86	0.717
gender.txt	0.00	100.00	0.5
opposites.txt	0.00	28.57	0.084

13.2 Comparison of first word to vector sum in relations

13.3 Discussion of Above Comparisons

In this analysis, we conducted two distinct comparisons to explore the relationships between word pairs in different semantic categories:

13.3.1 Methodology

1. In the first comparison, we directly compared the first word of each pair to the second word as they exist in the vector space.
2. In the second comparison, we added an average distance vector to the second word's vector and then compared this new vector to the first word. The average distance vector is the mean difference between the word vectors of the first and second words across all pairs in a given relationship category, representing the typical offset in the embedding space for that semantic relationship.

13.3.2 Results Analysis

Capitals and Countries

- Direct comparison: Low similarity (6.90% most similar, 24.14% in top 10)
- Vector sum method: Dramatic improvement (65.52% most similar, 86.21% in top 10)
- Mean Reciprocal Rank increased from 0.2002 to 0.806

This significant improvement suggests that the relationship between capitals and countries is consistently represented by a specific vector offset in the embedding space.

Demonyms

- Direct comparison: Low similarity (5.56% most similar, 44.44% in top 10)
- Vector sum method: Substantial improvement (65.71% most similar, 82.86% in top 10)
- Mean Reciprocal Rank increased from 0.223 to 0.717

Similar to capitals, the demonym relationship appears to be well-captured by a consistent vector offset.

Gender

- Direct comparison: No exact matches, 33.33% in top 10
- Vector sum method: Still no exact matches, but 100% in top 10
- Mean Reciprocal Rank increased from 0.264 to 0.5

While the vector sum method doesn't produce exact matches, it consistently brings the related gender term into the top 10, suggesting a more complex but still structured relationship.

Opposites

- Direct comparison: Relatively high similarity (28.57% most similar and in top 10)
- Vector sum method: Performance degraded (0% most similar, 28.57% in top 10)
- Mean Reciprocal Rank decreased from 0.513 to 0.084

This unexpected result suggests that opposite terms may not be related by a consistent vector offset, and their relationship in the vector space is more complex.

13.3.3 Key Insights

1. The vector sum method significantly improves prediction for geographical and cultural relationships (capitals and demonyms), indicating these relationships are consistently represented in the vector space.
2. Gender relationships show improvement in top 10 predictions but not in exact matches, suggesting a more nuanced representation.
3. The degradation in performance for opposites highlights the complexity of antonymic relationships in vector space, which aren't captured by simple vector addition.

13.3.4 Implications

These results demonstrate that different types of semantic relationships are encoded differently in the word embedding space. While some relationships (like geographical associations) can be effectively captured and predicted using vector arithmetic, others (like antonyms) require more sophisticated approaches. This understanding is crucial for developing more nuanced NLP models and for interpreting the limitations of current word embedding techniques.

13.4 Discussion of Similarity Across Multiple Runs

Repeated experiments with capitals.txt and opposites.txt revealed variations in results:

13.4.1 capitals.txt

- Analysis 1 (Direct Comparison):
 - Most Similar: 3% - 25%
 - Top 10: 39% - 50%
 - Mean Reciprocal Rank: 0.20 - 0.51
- Analysis 2 (Vector Sum Method):
 - Most Similar: 57% - 63%
 - Top 10: 86% - 93%
 - Mean Reciprocal Rank: 0.69 - 0.75

13.4.2 opposites.txt

- Both analyses:
 - Most Similar and Top 10: Nearly consistent
 - Mean Reciprocal Rank:
 - * Analysis 1: Variation range of 0.4
 - * Analysis 2: Variation range of 0.2

13.4.3 Key Observations

1. Data shuffling significantly affects Analysis 1 for capitals.txt, while Analysis 2 shows more stability but still notable variation.
2. For opposites.txt, Most Similar and Top 10 metrics are robust across shuffles, but Mean Reciprocal Rank shows sensitivity to data ordering in both analyses.
3. The Vector Sum Method (Analysis 2) generally provides more consistent results, particularly for capitals.txt, suggesting it may be a more reliable approach for certain types of semantic relationships.

These findings highlight the importance of multiple runs in evaluating word embedding models, as data ordering can impact results.

14 Lab 5: Word Sense Disambiguation and Decision Trees

Lexelt Questions

Exploring the Training Data

1. There are 57 different lexelts in the training data.

2. Part of speech breakdown of the 57 lexels:
 - Verbs: 32
 - Nouns: 20
 - Adjectives: 5
3. The lexelt "organization.n" has 112 instances (training examples).
4. Distribution of headwords and answers:
 - a) Number of headwords per instance: 1: 7709, 2: 143, 3: 6, 4: 1, 5: 1
 - b) Number of answers per headword instance: 1: 7204, 2: 645, 3: 9, 4: 2
5. The lexelt 'activate.v' has 4 unique sense IDs in the training data.
6. Random guessing accuracy on training data: 20.91% (0.2092)
7. Most Frequent Sense (MFS) baseline accuracy on training data: 55.34% (0.5534)
8. **Discussion:** Is the MFS baseline higher or lower than expected?
 The MFS baseline (55.34%) is significantly higher than random guessing (20.92%). This is higher than initially expected, implying that:
 - Most words have a dominant sense used more frequently in context.
 - The distribution of sense IDs is skewed, with one sense often dominating others in typical language use.
 - Word sense disambiguation may be easier than anticipated, as senses are not evenly distributed across words.

Exploring the Test Data

9. The lexelt 'activate.v' has 114 instances in the test data.
10. Comparison of senses in train and test data:
 - a) Senses in train but not in test: 1
 - b) Senses in test but not in train: 2
 - c) **Discussion:** A model trained without knowledge of new senses in the test data would struggle to make accurate predictions for those senses. However, extra senses in the training data not present in the test set would not directly harm the model's ability to classify known senses.
11. MFS accuracy on test data: 53.90%
12. Performance analysis:
 - Random Guessing: 20.92%

- MFS baseline (training): 55.34%
- MFS accuracy on test data: 53.90%

Discussion: The test accuracy of 53.90% aligns well with initial performance intuitions. It significantly outperforms random guessing (20.92%) as expected. The small performance drop (1.44%) from training to test suggests good generalization. The consistency between training and test results indicates similar word sense distributions in both datasets, bolstering the model’s reliability. This MFS baseline provides a solid benchmark for comparing more advanced word sense disambiguation techniques.

Decision Tree Questions

13) Comparing our accuracies:

- Decision Tree Classifier accuracy: 57.89%
- Most Frequent Sense (MFS) baseline accuracy: 54.03%

Yes, our decision tree with `random_state=0` does outperform the Most Frequent Sense (MFS) baseline.

The decision tree achieves an accuracy of 57.89%, which is 3.86 percentage points higher than the MFS baseline accuracy of 54.03%. This represents an improvement of approximately 7.14% relative to the baseline performance.

This result is encouraging because it demonstrates that:

1. The decision tree model is capable of learning meaningful patterns from the data that go beyond simply predicting the most frequent sense.
2. The features we’ve engineered and the context we’ve captured in our data representation are informative for the task of word sense disambiguation.
3. Even a relatively simple machine learning model like a decision tree can add value over a naive baseline in this complex NLP task.

While the improvement is modest, it’s a positive indication that our approach is on the right track. The fact that we can beat the MFS baseline, which is often quite strong in word sense disambiguation tasks, suggests that there’s potential for further improvements with more sophisticated models or feature engineering.

14) While deep learning methods generally outperform decision trees in complex NLP tasks, there are indeed situations where an NLP practitioner might prefer decision trees. Two important reasons are:

1. **Interpretability:** Decision trees provide a high level of interpretability that is often lacking in deep learning models. This is crucial in scenarios where:

- Stakeholders who are not well-versed in deep learning need to understand how decisions are made.
- The reasoning behind predictions needs to be explained in simple terms.
- Transparency in the decision-making process is required for regulatory compliance or ethical considerations.

With decision trees, one can easily visualize the decision path and understand which features are most influential in making predictions.

2. **Computational Efficiency:** Decision trees are generally less computationally demanding than deep learning models. This can be a critical factor in situations where:
 - Computational resources are limited, such as on mobile applications.
 - Real-time or near-real-time predictions are required, and the trade-off between slightly lower accuracy and faster processing is acceptable.
 - The dataset is relatively small, and the computational overhead of training a deep learning model isn't justified.
 - Rapid prototyping or experimentation is needed, allowing for quicker iteration and testing of ideas.

In these scenarios, the simplicity and efficiency of decision trees can be a significant advantage, allowing for practical deployment in resource-constrained environments or enabling faster development cycles.

These reasons highlight that while deep learning often provides superior performance, the choice of model should always consider the specific requirements and constraints of the task at hand. Decision trees, with their interpretability and efficiency, continue to be valuable tools in the NLP practitioner's toolkit for certain applications.

Random Forest Questions

15) Accuracies aggregated across all instances of all lexels:

- Decision Tree Classifier accuracy: 57.89%
- Most Frequent Sense (MFS) baseline accuracy: 54.03%
- Random Forest Classifier accuracy: 61.74%

Comparing these results:

1. **Random Forest vs. Decision Tree:** Yes, our Random Forest classifier with random state=0 outperforms the Decision Tree classifier. The Random Forest achieves an accuracy of 61.74%, which is approximately 3.85 percentage points higher than the

Decision Tree’s accuracy of 57.89%. This improvement is significant and aligns with our expectations, as Random Forests are ensemble models that typically perform better than individual decision trees due to their ability to reduce overfitting and capture more complex patterns in the data.

2. **Random Forest vs. MFS Baseline:** The Random Forest classifier also outperforms the Most Frequent Sense baseline. The Random Forest’s accuracy of 61.74% is 7.71 percentage points higher than the MFS baseline’s accuracy of 54.03%. This substantial improvement demonstrates that our machine learning approach is capable of capturing contextual information and making more informed predictions than simply guessing the most frequent sense.
3. **Decision Tree vs. MFS Baseline:** It’s worth noting that even the simpler Decision Tree classifier outperforms the MFS baseline by 3.86 percentage points. This indicates that both our machine learning approaches are adding value over the naive baseline strategy.

In conclusion, the Random Forest classifier provides the best performance among the three approaches, significantly outperforming both the Decision Tree classifier and the MFS baseline. This result highlights the effectiveness of ensemble methods like Random Forests in capturing the complex patterns necessary for word sense disambiguation tasks.

16) In our experimentation with hyperparameters, we observed the following:

1. **Random State:** Changing the random state did not significantly affect the model’s performance. This is expected, as the random state primarily affects the reproducibility of results rather than the overall performance of the model.
2. **Number of Estimators (n_estimators):** Reducing the number of estimators from 100 to 75 resulted in a slight increase in accuracy by approximately 0.12%. This minor improvement could be due to:
 - Reduced overfitting: Fewer trees might have prevented the model from capturing noise in the training data.
 - Better generalization: The reduced number of trees might have led to a more generalized model that performs better on unseen data.
3. **Max Features:** Changing max_features from "sqrt" to "None" led to a significant improvement in accuracy by 4%. This substantial increase can be attributed to:
 - Increased feature consideration: With "None", the model considers all features when making splits, potentially capturing more complex relationships in the data.
 - Better utilization of informative features: In word sense disambiguation, many features might be relevant, and considering all of them allows the model to make more informed decisions.
4. **Combined Effect:** When both changes (n_estimators = 75 and max_features = "None") were applied together, we observed a cumulative improvement of 3% + 0.12% = 3.12%, resulting in a final accuracy of 64.40%.

The best performance we achieved was 64.40% accuracy, obtained with the following hyperparameters:

- `n_estimators`: 75
- `max_features`: "None"

Why do these changes work? The improvement from using all features (`max_features` = "None") suggests that our word sense disambiguation task benefits from considering a wide range of contextual information. This aligns with the linguistic understanding that word senses often depend on subtle contextual cues that might be captured by various features.

The slight improvement from reducing the number of estimators indicates that our original model might have been slightly overfitting. The reduced number of trees helps in creating a more generalized model.

It's interesting to note that these hyperparameter changes had additive effects, suggesting that they address different aspects of model performance. This highlights the importance of considering multiple hyperparameters in combination when optimizing machine learning models for natural language processing tasks.

17) Some interesting Lexelts and their most important features:

1. **arm.n**

- "arm_of" (MDI: 0.1349)
- "political" (MDI: 0.0525)
- "service" (MDI: 0.0449)

The high importance of "arm_of" is expected, as it likely distinguishes between the anatomical sense and the figurative sense (e.g., "arm of the government"). The presence of "political" supports this interpretation, suggesting a frequent use in political contexts.

2. **climb.v**

- "roses" (MDI: 0.0936)
- "climbed_the" (MDI: 0.0313)
- "service" (MDI: 0.0280)

Surprisingly, "roses" is the most important feature for "climb.v". This could indicate a frequent use in gardening contexts (e.g., "climbing roses"). The presence of "climbed_the" is more expected, likely capturing typical usage patterns.

3. **eat.v**

- "Jay" (MDI: 0.1266)
- "will" (MDI: 0.0572)
- "skiing" (MDI: 0.0437)

The high importance of "Jay" is unexpected and might indicate a corpus-specific pattern or a named entity frequently associated with eating in the dataset. The presence of "skiing" is also surprising and could suggest a context where eating is discussed in relation to sports or leisure activities.

4. **party.n**

- "of" (MDI: 0.0905)
- "Mr" (MDI: 0.0553)
- "party" (MDI: 0.0541)

The importance of "of" could be related to phrases like "party of the first part" in legal contexts. "Mr" suggests frequent use in formal or news contexts, possibly referring to political parties. The presence of "party" itself as an important feature is expected and likely helps distinguish between different senses.

5. **different.a**

- "is_different" (MDI: 0.0348)
- "very_different" (MDI: 0.0281)
- "say" (MDI: 0.0268)

The importance of phrases like "is_different" and "very_different" is expected, as they are common collocations. The presence of "say" is interesting and might indicate that "different" is often used in reported speech or opinions.

Overall, we see a mix of expected and surprising features. Some features, like "arm_of" for "arm.n", align well with our intuitions about word senses. Others, like "roses" for "climb.v" or "Jay" for "eat.v", are more surprising and might reflect corpus-specific patterns or capture nuanced contexts that aren't immediately obvious.

The presence of function words (e.g., "of", "will") and punctuation as important features for some Lexelts suggests that syntactic structures play a role in disambiguating senses. This highlights the importance of considering not just lexical semantics but also syntactic patterns in word sense disambiguation tasks.

These results demonstrate the complexity of word sense disambiguation and the value of data-driven approaches in revealing linguistic patterns that might not be apparent through intuition alone. They also highlight the importance of carefully examining feature importance in understanding how machine learning models approach this task.

18) Overall, the transformation did not significantly improve our results. Our Random Forest accuracy with transformation was 64.10%, which is a slight decrease of 0.3% compared to the accuracy without transformation. This marginal difference suggests that the transformation had a minimal impact on the model's overall performance.

The impact of the transformation on our feature importance is particularly interesting: Analyzing these results, we can observe several interesting points:

Feature	Related, MDIV	...	...
arm.n	"arm of", 0.1328	"political", 0.0526	"service", 0.0422
climb.v	"roses", 0.0848	",", 0.0287	"more", 0.0275
eat.v	"Jay", 0.1224	"There", 0.0408	"skiing", 0.0335
party.n	"of", 0.1104	"party", 0.0594	"Mr", 0.0579
different.a	"different", 0.0472	"to", 0.0382	",", 0.0264

Table 1: Results after transformation

1. Context-specific features: The transformation seems to have highlighted more context-specific features. For example, "arm of" for "arm.n" could be indicative of its use in phrases like "arm of the government" or "arm of the sea", which are distinct from its anatomical meaning.
2. Unexpected important features: Some seemingly unrelated words have gained importance, such as "roses" for "climb.v" and "Jay" for "eat.v". This might indicate that the transformation is capturing some domain-specific or corpus-specific relationships that aren't immediately obvious.
3. Function words: The importance of function words like "of", "to", and commas has increased for some features. This could suggest that the transformation is giving more weight to syntactic structures.

The fact that these more nuanced features have emerged, yet the overall performance hasn't improved, could be due to several factors:

- Overfitting: The transformation might be causing the model to focus on very specific, potentially rare contexts, leading to overfitting on the training data.
- Feature sparsity: The transformation might have increased the sparsity of the feature space, making it harder for the model to generalize.
- Loss of generalization: While the transformed features capture more specific contexts, they might have lost some of the generalization capability that the original features had.

In conclusion, while the transformation has revealed interesting linguistic patterns and context-specific features, it hasn't translated into improved performance. This suggests that for this particular word sense disambiguation task, the original feature representation might already capture sufficient information for the model to make predictions.

15 Lab 6: POS Tagging (Tag Granularity, Vocabulary, and Document Size)

16 Exploring Tag Collapsing

For training on brown and testing on wsj,

Train Tag Set	Test Tag Set	Accuracy
PTB	PTB	89.67 percent
PTB	Universal	91.02 percent
Universal	Universal	90.61 percent

16.1 DISCUSS

We observed a clear pattern in our results when training on the Brown corpus and testing on WSJ. The accuracy increased with each test. From all PTB tags 89.67 percent, we see a slight increase to all Universal tags of 90.61 percent, with a further increase when training on PTB and evaluating on Universal tags to 91.02 percent. This pattern suggests that while more general tags, Universal, are beneficial, there's an advantage to training on more specific or granular tags like PTB, and then evaluating on broader categories like Universal. Contrary to our initial expectations, the more granular PTB tags performed worse than the less granular Universal tags. This indicates that for this task, and for our model, a more general set of tags leads to higher accuracy. The best performance came from the hybrid approach of training on PTB and evaluating on Universal, suggesting that the model benefits from learning 'fine-grained' distinctions during training, but performs better when making broader category distinctions during evaluation. The success of the PTB-to-Universal approach might be explained by the model's ability to learn detailed distinctions from PTB tags, while the Universal tag evaluation allows it to avoid potential over fitting or noise introduced by very specific tag categories. This hybrid method seems to strike a balance between detailed learning and generalized application. Despite the clear advantage of less granular tags in terms of overall accuracy, it's important to note that more granular tags might still be preferable in certain scenarios. For example, specific linguistic research, precise language processing in legal or medical fields, or applications requiring detailed grammatical analysis might benefit from more granular tags, even at the cost of some accuracy. The choice between granular and general tags in real-world applications should consider the specific requirements of the task, balancing the need for accuracy with the level of detail required.

Train Tag Set	Test Tag Set	Accuracy
PTB	PTB	92.32 percent
PTB	Universal	93.72 percent
Universal	Universal	92.87 percent

For training on wsj/train and testing on wsj/test,

16.2 DISCUSS

Switching to training and testing on the WSJ corpus we observed very similar patterns. The performance is consistent: with PTB tags only reporting 92.32 percent, Universal tags only reporting 92.87 percent, and PTB trained / Universal evaluated performed the best at 93.72 percent. Across all configurations we saw a 2-3 percent accuracy improvement across all three tests. This considerable increase in performance shows the importance of fitting training data to test data.

Again, the hybrid approach, PTB trained and Universal evaluated performed the best, but the gap (from previous results) narrowed. This suggests that when training and testing on the same data, PTB has less influence when used for training. The consistency of accuracy's strengthens our conclusions that training on more granular tags and evaluating on more general ones is most beneficial. The consistent benefit of the hybrid approach alludes to the importance of capturing fine-grained distinctions during training while allowing for more flexible, broader categorizations during the evaluation stage. This suggests that our model benefits from learning nuanced, specific, differences between tags, but performs better when it has some leeway in making final classifications."

17 Exploring Vocabulary Size Effects

Vocab Size	Train Time	Test Time	Model Size	Accuracy
1000	1m2.007s	2m6.050s	model1000.pkl	82.78 percent
2000	1m20.511s	2m13.631s	model2000.pkl	85.68 percent
5000	2m2.131s	2m42.015s	model5000.pkl	88.79 percent
10000	2m48.889s	3m2.223s	model10000.pkl	90.57 percent
20000	3m47.813s	3m44.810s	model20000.pkl	91.79 percent
50000	5m8.014s	4m45.288s	model50000.pkl	91.02 percent

17.1 DISCUSS

As vocabulary size increases there's a noticeable improvement in accuracies. From 82.72 percent at 1000 words to 91.79 at 20000 words. One thing that we found surprising was that accuracy slightly decreased when moving from 20000 to 50000 words.

The most dramatic improvements were observed in early increases of vocab size, particularly from 1000 to 5000 words. This rate of improvement slows as the vocabulary size grows, suggesting diminishing returns for larger vocabularies.

It's important to note the relationship between vocabulary size and computational resources. Both training and testing times increase substantially as the vocabulary grows. For instance, training time increases from about 1 minute for 1000 words to over 5 minutes for 50000 words. Similarly, testing time rises from about 2 minutes to nearly 5 minutes. This trade-off between accuracy and computational cost is a crucial consideration in model design.

The slight decrease in accuracy at 50000 words could be due to several factors. While it might suggest overfitting, we'd need more data to confirm this hypothesis. It could also be a result of introducing rare words that add noise to the model without contributing significantly to overall accuracy. Alternatively, it might indicate that our current model architecture or training process is not optimally suited for very large vocabularies.

In conclusion, while larger vocab sizes tend to increase accuracy, there's a point of diminishing returns, and potentially even performance degradation. The optimal vocabulary size for this task appears to be around 20000 words, balancing accuracy gains with computational costs. However, the choice of vocabulary size in practice would depend on specific project requirements, including desired accuracy, available computational resources, and time constraints.

Exploring Document Size Effects

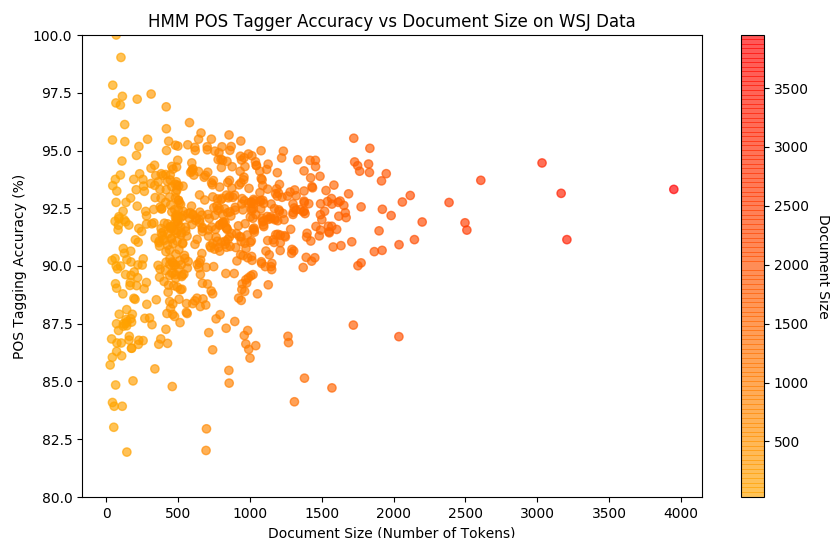


Figure 7: POS Tagger Accuracy vs Document Size

17.2 DISCUSS

We observed a clear trend of increasing accuracy as document size increases. The range always is between 80-100 percent so we decided to zoom in on that part of the plot. We observed that as document size increases, the lower bound of accuracy rises and the spread narrows. This points to a gradual increase in both accuracy and consistency as token size increases. There is some variance. Small documents (0-500 tokens) have the widest range of accuracies ranging from 80-100 percent. This is due to many factors, one for example is that each word carries more weight in smaller docs, so one mis-tag has a larger impact on the accuracy percentage. Likewise, smaller documents have fewer tokens to tag, and if a document is very small, there's a slight chance that our model will tag everything correctly by chance alone. Larger documents provide more context to the taggers which lead to more consistent and accurate tagging. We observed that for documents larger than 2000-2500 tokens, accuracy consistently stays above 90 percent. There is some over-plotting, as there are more available data points for smaller sized documents so the data overlaps. This overplotting of small documents reinforces that small documents have higher variance—shown as small documents (0-500) plot anywhere from 80 percent to 100 percent. In conclusion, this plot does a wonderful job demonstrating the relationship between document size and part of speech tagging accuracy. Overtime the lower bound of accuracy increases and the spread narrows, which shows a gradual increase in part of speech tagging accuracy as document size grows. The high variability in small documents highlights the challenges and variance of tagging speech from shorter texts (tweets?). This tagger performs well with any data size (above 80 percent), but it becomes more consistent and more accurate as its fed more information.

18 Lab 7: Parsing XML, Sparse Matrices, and PCL Detection

Parsing XML Files

Question 1

The two functions "do xml" and "short xml" do the same thing fundamentally in that they both parse XML files. The former is more robust and scalable and the latter more suitable for small, hobby XMLs.

Because of "do xml" does not load in the entire file like "short xml" does, its memory usage is far more efficient, meaning it can handle longer XML files more effectively because only parts of the file are being fed into the program. It uses yield elements matching a specific tag, which is far more memory efficient than returning them. It also clears the yield variable after each time a match is found which is great for memory efficiency.

Conversely, "short xml" returns all matching entries at once. This is more memory intensive and can create long periods of unknowing if XML files are large.

Sparse Matrices

Question 2

For approximately 10,000 examples each containing 10,000 features that are each 1 byte it will take 100MB of storage space would be needed to store the feature matrix in memory

Question 3

Data Storage in LIL Matrix

The data in a LIL (List of Lists) matrix is stored in two arrays:

- **Data:** A list of lists where each inner list corresponds to a row in the matrix and contains the non-zero values in that row.
- **Rows:** A list where each element corresponds to a row and contains the column indices of the non-zero elements in that row.

Storage Requirements

- **1% Non-Zero Entries:** If only 1% of the entries in a LIL matrix are non-zero (assuming the training data was the same as in Question 2), then it will take about 3 MB of storage (1 MB for the data array, 2 MB for the rows array).
- **10% Non-Zero Entries:** If 10% of the entries in the LIL matrix are non-zero, then it will take about 30 MB of storage (10 MB for the data array, 20 MB for the rows array).

Crossover Point Between LIL Matrix and Dense Array

The storage requirement for the LIL matrix can be estimated as:

$$\text{LIL matrix size} = 100,000,000 \times x \times 3$$

where x is the proportion of non-zero elements.

Solving for when the LIL matrix equals the dense array size, we find the crossover at 30% non-zero elements. Above this proportion, a normal NumPy array would be more memory efficient. It is worth noting that there will be object overheads that slightly increase memory usage in practice.

Performance Considerations

With a LIL matrix, certain mathematical operations will be slow. For example:

- **Matrix-Matrix Multiplication:** This operation would be inefficient because of the linked list format.
- **Column Access:** Any operation that involves accessing columns will be slow because the LIL format is optimized for row access.

Analysis

Question 4

Classifier Performance Comparison

Classifier	Accuracy	Precision	Recall	F1 Score
MultinomialNB	0.8696	0.2257	0.1552	0.1840
DummyStratified	0.8699	0.2227	0.1503	0.1794
DummyMostFrequent	0.8680	0.2146	0.1483	0.1754

Table 2: Performance comparison of classifiers on the PCL detection task

Analysis

The `MultinomialNB` classifier achieves the highest F1 score of **18.40%** among the three classifiers, but the difference is marginal (`DummyStratified`: 17.94%, `DummyMostFrequent`: 17.54%). This suggests that the Bag-of-Words features are only marginally more informative than random guessing for this task. It also indicates that the dataset is imbalanced, with a much larger number of non-condescending examples than condescending ones.

Precision and recall are low across the board, which indicates difficulty in correctly identifying positive cases of condescending language.

Question 5

Condescending Example 1

id="@@14317261" category="homeless" country="bd" condescension="true" score="4"

"SM Hassan Siddiquee Pope Francis has proclaimed Mother Teresa of Kolkata a saint, hailing her as the personification of maternal love and a powerful advocate for the poor. Queen Sofia of Spain and some 1,500 homeless people also looked on as Pope Francis described Saint Teresa's work in the slums of the Indian Metropolis as 'eloquent witness to God's closeness to the poorest of the poor.' There has been debates and controversies over Saint Teresa's legacy since she ..."

For this example, our classifier is confident because the repeated mentions of "the poor," "homeless people," and "slums" might be triggering features that the classifier has associated with condescending language. This example is extremely condescending because of the juxtaposition it creates between houseless people and that of nobility is harsh and could be viewed as offensive.

Condescending Example 2

id="@@15842106" category="hopeless" country="ng" condescension="false" score="0"

"Afejuku uses the poem, 'A Leaping Flame' to expose the belief among the Itsekiri that their land was stolen by those who have high connection in government. The poem is dedicated to the memory of Pa J. O. S Ayomike, a frontline Itsekiri activist and historian who fought for the sovereignty of the Itsekiri nation. In the poem, the speaker re-echoes the words of Pa Ayomike: 'Nothing is final, he chants. No one shall see the end/Until the land-owners of Oeri regain and retain their lands/Stolen along the corridors and in the beds of love.' Like several poems in Afejuku's collections, there is the lingering hope that one day the Itsekiri nation will regain their 'stolen' lands. A tone of resolute defiance runs through Afejuku's poems. It constitutes a point of equi-balance to what might otherwise have been a hopeless view of life."

For this example, our classifier is confident likely because of the part about stolen lands and hopelessness. It is also interesting how this example also contains a quote. This example is seemingly not condescending because its tone makes it seem as though it is a historical account about the Itsekiri nation. Also, the annotators marked it as not condescending.

Condescending Example 3

id="@@16183055" category="hopeless" country="ph" condescension="true" score="3"

"During these stressful times, our country is enveloped with more shadows than light, more darkness of fear, doubts, hopelessness, and anxiety with the rising cases of extrajudicial killings, drug addictions, and acts of terrorism. People are living in darkness. There is an impending crisis in our country... crisis of FAITH and a crisis of HOPE"

For this example, our classifier is confident likely because of the excess of words that naturally carry condescending implications (e.g., killings, drug, addictions, terrorism, darkness, crisis...). This example is condescending due to its negative outlook and potential to be the result of certain groups.

Non-Condescending Example 1

id="@@10110089" category="disabled" country="tz" condescension="false" score="0"

"By continuing to use this site you consent to the use of cookies on your device as described in our cookie policy unless you have disabled them. You can change your cookie settings at any time but parts of our site will not function correctly without them. Close"

For this example, our classifier is confident because there are no parts of this text that might have clear condescending associations. The model's prediction is right as this example is not condescending.

Non-Condensing Example 2

id="@@14806682" category="poor-families" country="ie" condensation="false" score="0"

"Each year, 15 million girls are married before the age of 18. In developing countries one in three girls is married before the age of 18 and one in nine before the age of 15. * In Dominican Republic, 37% of women aged 20–24 are married before 18 years. * In most countries, girls from poor families are more likely to be married early than their richer peers. In Nigeria, 40% of the poorest girls are married by age 15 compared to 3% of the richest girls. * Girls in particular regions of some countries are disproportionately affected. In Ethiopia more than half of girls in the Amhara, Benishangul Gumuz and Afar regions marry by age 18 compared to 12% in Addis Ababa. The majority of the 25 countries with the highest rates of child marriage are considered fragile states or at high risk of natural disaster."

For this example, our classifier is confident because there is no heavy usage of condensing language. The model's predictions are correct as this example is annotated as not condensing as well.

Non-Condensing Example 3

id="@@23754955" category="migrant" country="au" condensation="true" score="4"

"Most are from desperately poor Horn of Africa countries and like tens of thousands each year, were willing to risk the treacherous journey through war-torn and impoverished Yemen in the hope of finding work in Saudi Arabia and other wealthy Gulf Arab states. But their plan was not to be. Caught and detained by the Yemeni authorities, the 600 or so men now await deportation, prevented from leaving their makeshift jail by armed soldiers. Conditions at the warehouse are growing increasingly desperate. Several days ago, the authorities stopped handing out food and basic supplies. 'I came from Djibouti to work. They used to give us small amounts of food. If there is no food, we will die,' said one migrant who declined to give his name. 'If there is no solution, they will deport us to our countries or get us out of here,' the man added. Yemen, the poorest country on the Arabian Peninsula, has long been a transit point for migrants and refugees from East Africa, many of whom are fleeing hunger and violence. The route was unsafe long before Yemen descended into all-out war in 2015. Hundreds drown each year on the perilous sea journey, and those who make it face multiple risks, including death by 'assault, extortion and abuse by criminal networks, as well as the hazards stemming from the war in Yemen itself,' according to UNHCR, the United Nations' refugee agency. Despite these dangers, the number of migrants traveling across the Gulf of Aden and Red Sea to Yemen has been rising steadily, to 117,000 in 2016 from 92,000 in 2015, according to UNHCR. Khaled al-Elwany, a local official who until recently oversaw the deportation of migrants back to their home countries, says up to 15,000 migrants are arrested at city checkpoints

each month. Elwany says he was fired by the interior ministry for refusing to cooperate with a plan to relocate the center to a neighboring province and after authorities stopped food supplies. A ministry official told Reuters that the aim of the move to a new facility had been to improve conditions for the migrants while they await deportation. Saudi Arabia and a coalition of mostly Gulf Arab allies are fighting to restore Yemen's exiled government to power and roll back the Iran-aligned Houthi movement. While the government nominally controls Aden and other southern areas, a budget crunch and rivalries with local armed groups mean its officials mostly reside abroad. State services and salaries have been hit hard by the chaos, and the African migrants appear to be a lower priority. The frightened men in the Aden warehouse, subsisting on bread bought with meager savings and washing their clothes in an rubbish-strewn outdoor space, are just the tip of the iceberg."

For this example, our classifier is confident because there are few words that could be condescending. It is wrong because this text is condescending towards migrants in patronizing ways (e.g., desperately poor, makeshift jail). They are made out to have little agency.

Question 6

These examples are challenging for my Naive Bayes classifier due to their ambiguity and lack of clear condescending indicators:

Example 1

Confidence: 0.5001

id="@@16101010" category="in-need" country="au" condescension="false" score="0"

"Role: The Game (A festival-wide contest of silliness and creativity) is in need of a villain and in my generosity and my humbleness I have stepped up to become the Evilord."

Classifier difficulty: Playful language misinterpreted as potentially condescending.

Human perspective: Easily recognizable as humorous, not condescending.

Key aspect: Sarcasm and context, which the classifier can't detect.

Example 2

Confidence: 0.5020

id="@@2863218" category="in-need" country="ph" condescension="false" score="0"

"GMA Kapuso Foundation reaches out to thousands in need"

Classifier difficulty: Neutral tone with 'in need' phrase causing uncertainty.

Human perspective: Clearly non-condescending, factual reporting.

Key aspect: Understanding of charitable actions and neutral reporting tone.

Example 3

Confidence: 0.5026

id="@@19934435" category="hopeless" country="nz" condescension="false" score="0"

"Some fans across America woke up to a newspaper on their doorstep that features some clues and hints about hopeless fountain kingdom. **26;452;TOOLONG"

Classifier difficulty: 'Hopeless' might be misinterpreted as condescending.

Human perspective: Recognizable as a neutral statement about fan engagement.

Key aspect: Knowledge that 'hopeless fountain kingdom' is likely an album title, not a literal description.

These examples highlight the limitations of simple text classification models in capturing the nuances of human language, especially with short, ambiguous texts. While humans can easily interpret these statements using broader contextual understanding, the classifier struggles with the lack of clear linguistic markers of condescension.

Question 7

Classifier Performance Comparison

Classifier	Accuracy	Precision	Recall	F1 Score
MultinomialNB	0.8432	0.2041	0.2264	0.2147

Table 3: Performance with `fit_prior=False` on MultinomialNB

Analysis

Using the class priors from training data was beneficial for overall accuracy. The model had more false positives with class priors set to false. The model was able to identify more actual positive cases. The F1 score increased, which indicates a better balance between precision and recall.

Conclusion

If overall accuracy is important, stick with the default settings. If identifying more potentially condescending texts is crucial, even at the cost of more false positives, the `fit_prior=False` configuration might be preferable.

Question 8

Additional Features to Improve the Classifier

Three features we could extract to better our current configuration are:

1. **N-grams (bigrams and trigrams):** Phrases like "obviously you don't understand" or "let me explain simply" are strong indicators of condescension that would be missed by a unigram model like ours.
2. **POS tags:** The frequent use of second-person pronouns (*you*) followed by negative verbs or adverbs might indicate condescension.
3. **Sentiment score:** Condescending language often involves a mix of positive and negative sentiment. By including overall sentiment polarity as a feature, we can capture nuanced emotional tones that are often present in condescending text but might be missed by simple word frequency analysis.

Expanding to Categories

Question 9

Examples Where the Classifier is Overconfident

The classifier is overconfident with the following examples:

Example 1

*ID: @@1859232 Confidence: 0.5000 Label: **False***

"Traditionally, most vulnerable Tanzanians tend to rely on their families and on informal networks as a mechanism for coping with shocks and insecurity."

Classifier difficulty: Presence of the word "vulnerable" may trigger condescension detection.

Human perspective: Recognizes the statement as neutral and informative.

Reasoning: The term "vulnerable" is used descriptively without a condescending tone.

Example 2

*ID: @@25857187 Confidence: 0.5014 Label: **False***

"Ndii insists that Muthiora has been investigating his family and attacking vulnerable people in his family. 'My mum has actually been confronted by people and this is unfortunate. They came spying on my mother. I have a duty to protect the privacy of my family,' said Ndii."

Classifier difficulty: The word "vulnerable" may be misinterpreted as condescending.

Human perspective: Sees the text as a personal account without condescension.

Reasoning: The narrative focuses on personal experiences and concerns.

Example 3

ID: @@4601692 Confidence: 0.5015 Label: **False**

"We want to specifically focus on the youth, especially since our children are most vulnerable to tobacco use,' Dr. Ferguson stated."

Classifier difficulty: The phrase "most vulnerable" might flag potential condescension.

Human perspective: Understands the concern for youth without detecting condescension.

Reasoning: The statement expresses concern in a factual manner.

Analysis

The classifier finds these examples difficult to categorize for several reasons:

- **Presence of the word "vulnerable":** All three examples contain the word "vulnerable," which might be associated with condescending language in some contexts. However, the use of this word alone is not sufficient to determine condescension.
- **Lack of explicit condescending language:** These examples don't contain obviously condescending phrases or tone, making them challenging for a simple bag-of-words model.
- **Neutral or factual tone:** All three examples present information in a relatively neutral or factual manner, which doesn't align clearly with condescending or non-condescending categories.

Human Interpretation

As humans, we don't find these examples particularly condescending because we can consider:

- **Context and intent:** We understand that these statements are reporting on vulnerable populations or discussing vulnerability without adopting a condescending tone.
- **Sentence structure and overall meaning:** We can interpret the full meaning of the sentences, not just individual words.
- **Tone and sentiment:** We can detect the neutral or concerned tone in these statements, which differs from a condescending tone.

Question 10

Analysis of Holdout vs. Cross-Validation Results

The results above indicate the holdout example is slightly more likely to have extremely uncertain examples.

Advantages of Cross-Validation

Cross-validation provides insights across different categories, offering a more general view of the model's performance.

Advantages of Holdout Method

The holdout method gives specific insights into how the model performs on the "vulnerable" category, which could be valuable if this category is of particular interest.

Conclusion

We did not find our results here to be surprising because the variance between results is minor.

Question 11

Category	Accuracy	Precision	Recall	F1 Score
Vulnerable	0.6710	0.1509	0.7108	0.2489
Disabled	0.5968	0.1227	0.7407	0.2105
Homeless	0.5993	0.2416	0.6971	0.3588
Hopeless	0.2771	0.1494	0.9697	0.2588
Women	0.6493	0.0932	0.6852	0.1641

Table 4: Performance Metrics by Category

There is some variance between different held out sets. This is expected because different condescending categories tend to occur in the context of other potentially condescending words.

Question 12

Conclusion

The task of categorizing PCL (Patronizing and Condescending Language) is not well-defined. As we have learned throughout this block, natural language is nuanced and context-dependent—two factors that are very hard to detect using simple classifiers like ours. It is incredibly challenging to determine whether language is condescending because the definition of condescension is largely subjective. Furthermore, condescending language is often communicated through subtle cues like sarcasm.

In our opinion, simple models like ours are a step toward understanding condescension, but many additional capabilities must be added for condescension to be detectable on par with human perception.